LAPORAN JOBSHEET
“18 -Optimasi Performa Aplikasi Menggunakan Fitur Next.js”
MATA KULIAH PEMROGRAMAN BERBASIS FRAMEWORK

M. FATIH AL GHIFARY

2341720194 / TI-3A

PROGRAM STUDI D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

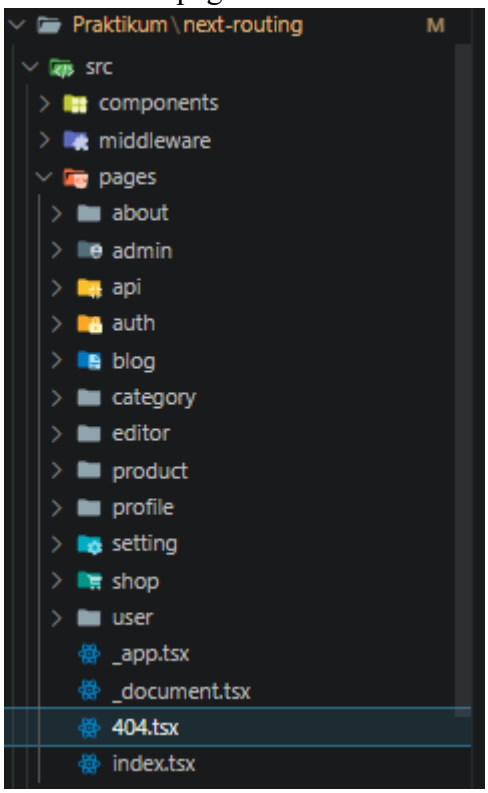
POLITEKNIK NEGERI MALANG

2026

PRAKTIKUM 1 – IMAGE OPTIMIZATION

PRAKTIKUM 1 – IMAGE OPTIMIZATION

A. Optimasi Gambar Lokal (Public Folder)
   Studi Kasus: Mengganti tag <img> pada halaman 404 dengan next/image.
   - Buka file src/pages/404.tsx



   - Modifikasi bagian imagenya

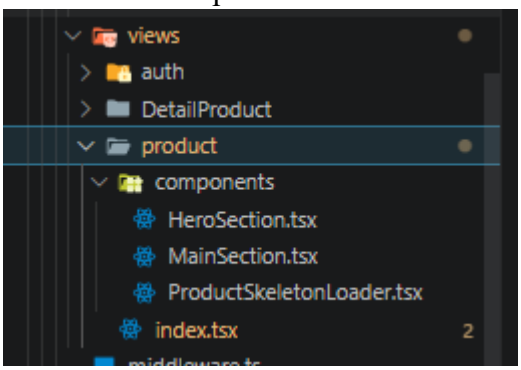
```
import Head from "next/head";
import Image from "next/image";
import Link from "next/link";
import styles from "../styles/404.module.scss";

const Custom404 = () => {
  return (
    <>
      <Head>
        <title>404 - Halaman Tidak Ditemukan</title>
      </Head>
      <div className={styles.error}>
        <Image
          src="/page-404.png"
          alt="404"
          width={400}
          height={400}
          className={styles.error__image}
          priority
        />
```

   Hasil:
   - Warning hilang
   - Image dioptimasi otomatis
   - Mengurangi bandwidth

- Mendukung lazy loading otomatis

B. Optimasi Gambar Remote (External URL)

1. Buka file views/product/index.tsx



2. Modifikasi file index.tsx

```
<div className={styles.product_card}>
  <Image
    src={product.image}
    alt={product.name}
    className={styles.product_image}
    width={0}
    height={0}
    sizes="100vw"
    style={{ width: "100%", height: "auto" }}
  />
```

dikarenakan gambar diambil dari url tertentu maka konfigurasi berbeda

3. Buka file next.config.js

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  images: {
    remotePatterns: [
      {
        protocol: "https",
        hostname: "cdn.shopify.com",
      },
      {
        protocol: "https",
        hostname: "lh3.googleusercontent.com",
      },
      {
        protocol: "https",
        hostname: "seekindonesia.com",
      },
      {
        protocol: "https",
        hostname: "dynamic.zacdn.com",
      },
      {
        protocol: "https",
        hostname: "footlocker.id",
      },
      {
        protocol: "https",
        hostname: "cdn-images.farfetch-contents.com",
      },
      {
        protocol: "https",
        hostname: "encrypted-tbn0.gstatic.com",
      },
    ],
  },
};

module.exports = nextConfig;
```

4. Hasil:

- Gambar di-proxy melalui /_next/image
- Performa lebih optimal
- Kompresi otomatis

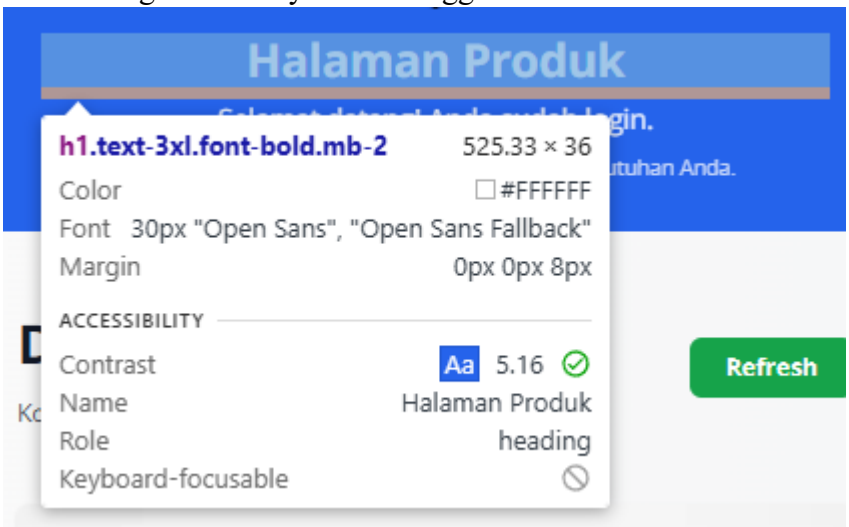
PRAKTIKUM 2 – FONT OPTIMIZATION

A. Menggunakan next/font

1. Buka file index.tsx pada folder Appshell/index.tsx dan modifkasi

```
P2 > Praktikum > next-routing > src > components > layout > Appshell > index.tsx > ...
import Navbar from "../navbar";
import Footer from "../footer";
import { useRouter } from "next/router";
import { Open_Sans } from "next/font/google";

const disableNavbar = ["/auth/login", "/auth/register", "/404"];
const disableFooter = ["/auth/login", "/auth/register", "/404"];

type AppshellProps = {
  children: React.ReactNode;
};

const openSans = Open_Sans({
  subsets: ["latin"],
  weight: ["400", "500", "700"]      Ctrl+I Windsurf Command
});

const Appshell = ({ children }: AppshellProps) => {
  const { pathname } = useRouter();
  console.log("Current Path:", pathname);  ᵇ 'Current Path:' | '/product'
  return (
    <main className={openSans.className}>
      {!disableNavbar.includes(pathname) && <Navbar />}
      {children}
      {!disableFooter.includes(pathname) && <Footer />}
    </main>
  );
};

export default Appshell;
```

2. Jalankan browser localhost:3000/product maka font akan berubah menjadi roboto untuk mengecek fontnya bisa menggunakan extension FontFinder



3. Hasil:
   - Tidak perlu load dari CDN manual
   - Tidak blocking render
   - Performance meningkat
   - Tidak terjadi FOUT (Flash of Unstyled Text)

PRAKTIKUM 3 – SCRIPT OPTIMIZATION

A. Menggunakan next/script

1. Buka file index.tsx pada folder layouts/Navbar dan modifikasi

```
5   import Script from "next/script";
6
7   const Navbar = () => {
8     const { data: session } = useSession();
9     return (
10      <div className={styles.navbar}>
11        <div className={styles.navbar__brand} id="title">
12          <Script id="title-script" strategy='lazyOnload'>
13            {`document.getElementById("title").innerHTML = 'My App';`}
14          </Script>
15        </div>
```

2. Pada kasus diatas kita merubah line 11 menggunakan model Typescript dapat terlihat ketika kita refresh web produk tulisan myApp akan terlihat berkedip
3. Perbedaan mendasar antara Line 11-13 dan Line 15-18 pada file index.tsx Anda terletak pada metode rendering teks dan interaksi dengan DOM (Document Object Model). Berikut adalah rincian perbedaannya:
   I. Metode Rendering
      a. Line 11-13 (Standard React/JSX): Ini adalah cara standar React. Teks "MyApp" ditulis langsung di dalam tag div. React akan langsung merender teks ini ke dalam HTML saat komponen dimuat.
      b. xLine 15-18 (Next.js Script & Manual DOM): Menggunakan komponen <Script> dari Next.js dengan strategi lazyOnload. Teks tidak ada di dalam HTML saat awal dimuat, melainkan baru "disuntikkan" secara manual menggunakan perintah JavaScript document.getElementById('title').innerHTML = 'MyApp'; setelah script tersebut diunduh di latar belakang
   II. Performa dan SEO
      a. Line 11-13: Sangat baik untuk SEO karena teks "MyApp" langsung terbaca oleh robot pencari (crawler) dalam kode sumber HTML
      b. Line 15-18: Kurang baik untuk SEO untuk konten penting karena teks baru muncul setelah JavaScript dijalankan. Strategi lazyOnload berarti script ini dijalankan paling akhir setelah semua sumber daya utama selesai dimuat, sehingga mungkin ada jeda waktu (delay) sebelum teks muncul di layar.
   III. Keamanan (XSS)
      a. Line 11-13: Aman karena React secara otomatis melakukan escape pada string untuk mencegah serangan Cross-Site Scripting (XSS).

b. Line 15-18: Memiliki risiko keamanan lebih tinggi karena menggunakan .innerHTML. Jika data yang dimasukkan berasal dari input user, ini bisa dimanfaatkan untuk menyuntikkan skrip berbahaya.

4. Hasil:
   - Script tidak blocking
   - Cocok untuk Google Analytics
   - Performa lebih ringan

PRAKTIKUM 4 – OPTIMASI AVATAR DENGAN NEXT/IMAGE

1. Buka file index.tsx pada folder layouts/navbar dan modifikasi

```
import Image from "next/image";
```

```
<Image
  src={
    session.user.image ||
    `https://ui-avatars.com/api/?name=${encodeURIComponent(
      session.user.name ||
        session.user.fullName ||
        session.user.email ||
        "User",
    )}&background=random&color=fff`
  }
  alt={session.user.fullName || "User Avatar"}
  width={45}
  height={45}
  className={styles.navbar__avatar}
/>
```

2. Tambahkan hostname Google

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  images: {
    remotePatterns: [
      {
        protocol: "https",
        hostname: "cdn.shopify.com",
      },
      {
        protocol: "https",
        hostname: "lh3.googleusercontent.com",
      },
```

TUGAS PRAKTIKUM

1. Optimasi semua image di project menggunakan next/image
   - Navbar (image avatar)

```
<div className={styles.navbar__user_container}>
  <div className={styles.navbar__user_info}>
    <Image
      src={
        session.user.image ||
        `https://ui-avatars.com/api/?name=${encodeURIComponent(
          session.user.name ||
            session.user.fullName ||
            session.user.email ||
            "User",
        )}&background=random&color=fff`
      }
      alt={session.user.fullName || "User Avatar"}
      width={45}
      height={45}
      className={styles.navbar__avatar}
    />
```

   - Product Image

```
div key={product.id} className={styles.produ
  <div className={styles.product_card}>
    <Image
      src={product.image}
      alt={product.name}
      className={styles.product_image}
      width={400}
      height={400}
    />
```

2. Gunakan minimal 1 font dari next/font

```
import Navbar from "../navbar";
import Footer from "../footer";
import { useRouter } from "next/router";
import { Open_Sans } from "next/font/google";

const disableNavbar = ["/auth/login", "/auth/register", "/404"];
const disableFooter = ["/auth/login", "/auth/register", "/404"];

type AppshellProps = {
  children: React.ReactNode;
};

const openSans = Open_Sans({
  subsets: ["latin"],
  weight: ["400", "500", "700"]      Ctrl+I Windsurf Command
});

const Appshell = ({ children }: AppshellProps) => {
  const { pathname } = useRouter();
  console.log("Current Path:", pathname);  ×20 ᵇ 'Current Path:' | '/editor'
  return (
    <main className={openSans.className}>
      {!disableNavbar.includes(pathname) && <Navbar />}
      {children}
      {!disableFooter.includes(pathname) && <Footer />}
    </main>
  );
};

export default Appshell;
```

3. Tambahkan script Google Analytics menggunakan next/script

```
P2 > Praktikum > next-routing > src > components > layout > navbar > index.tsx > Navbar
import styles from "./navbar.module.css";
import { signIn, signOut, useSession } from "next-auth/react";
import Image from "next/image";
import Link from "next/link";
import Script from "next/script";

const Navbar = () => {
  const { data: session } = useSession();
  return (
    <div className={styles.navbar}>
      <div className={styles.navbar__brand} id="title">
        <Script id="title-script" strategy='lazyOnload'>
          {`document.getElementById("title").innerHTML = 'My App';`}
        </Script>
      </div>
```

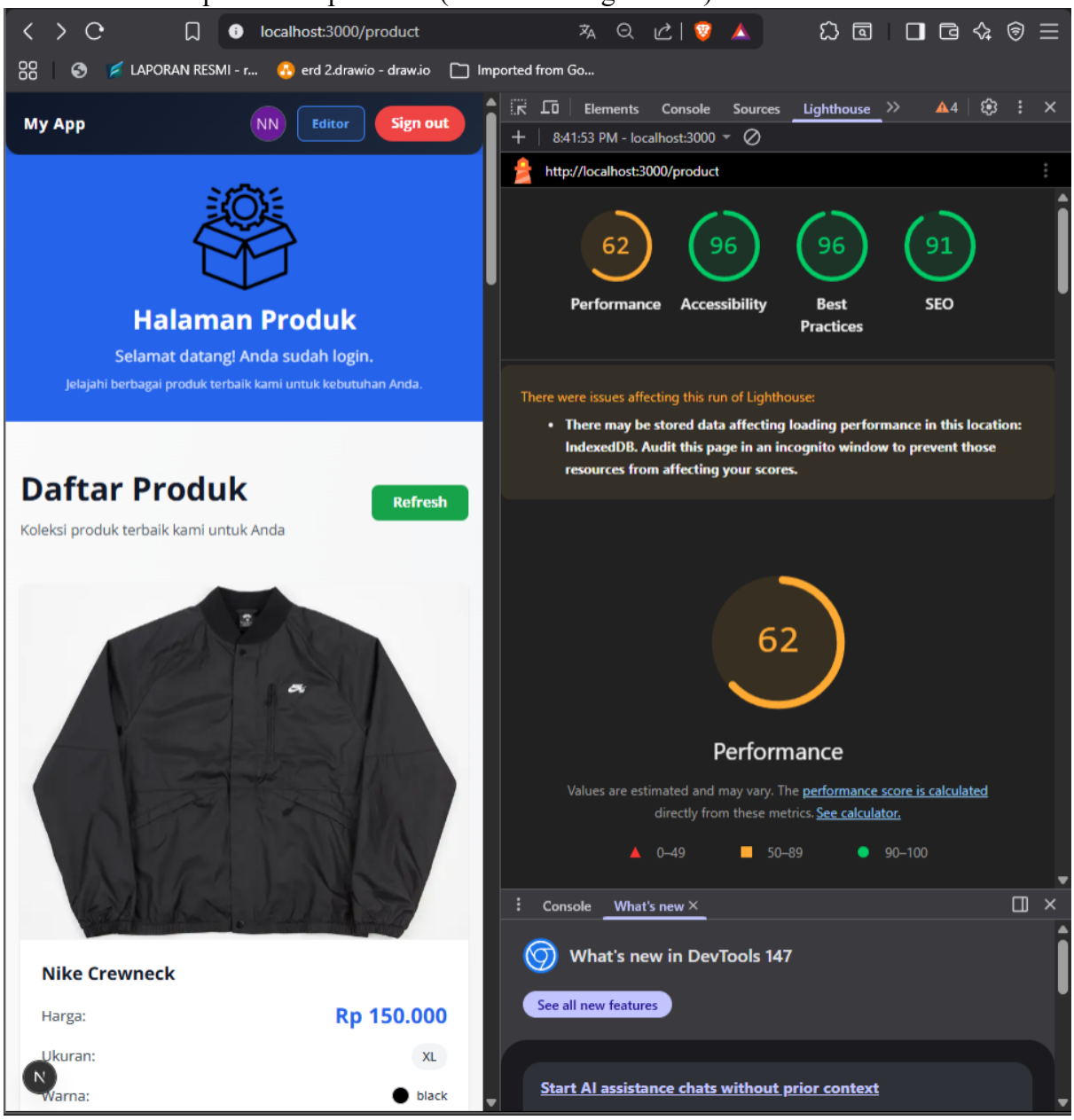
4. Terapkan dynamic import pada minimal 1 komponen
   - Pages/product/index.tsx

```
P2 > Praktikum > next-routing > src > pages > product > index.tsx > ...
import dynamic from "next/dynamic";
import ProductPageLoading from "../../views/product/components/ProductPageLoading";

const ProductPageContent = dynamic(
  () => import("../../views/product/components/ProductPageContent"),
  {
    loading: () => <ProductPageLoading />,
    ssr: true,
  },
);

const ProductPage = () => {
  return <ProductPageContent />;
};

export default ProductPage;
```

   - Pages/editor/index.tsx

```
P2 > Praktikum > next-routing > src > pages > editor > index.tsx > ...
import dynamic from "next/dynamic";
import EditorPageLoading from "../../views/editor/EditorPageLoading";

// Dynamic import dengan ssr: false mencegah rendering di server-side
const EditorPageContent = dynamic(
  () => import("../../views/editor/EditorPageContent"),
  {
    loading: () => <EditorPageLoading />,
    ssr: false,
  },
);

export default EditorPageContent;
```

5. Dokumentasikan perubahan performa (screenshot Lighthouse)



REFLEKSI DAN DISKUSI

1. **Mengapa <img> biasa tidak optimal?**
   Penggunaan tag <img> standar dianggap tidak optimal dalam pengembangan web modern, khususnya pada framework seperti Next.js, karena beberapa alasan teknis:
   - **Pemuatan Serentak (Eager Loading):** Tag <img> akan mencoba mengunduh seluruh gambar yang ada di halaman saat pertama kali dimuat, meskipun gambar tersebut berada di bagian bawah yang belum terlihat oleh pengguna. Hal ini membuang bandwidth dan memperlambat kecepatan akses awal.
   - **Kurangnya Optimasi Ukuran:** Gambar yang disajikan lewat <img> memiliki ukuran file yang tetap. Jika kamu mengunggah gambar beresolusi 4K namun menampilkannya di layar ponsel kecil, browser tetap harus mengunduh file besar tersebut, yang sangat tidak efisien.
   - **Pergeseran Tata Letak (Cumulative Layout Shift):** Tanpa definisi ukuran yang ketat, browser seringkali tidak tahu berapa ruang yang harus disediakan untuk gambar sebelum file selesai diunduh. Akibatnya, konten teks di bawahnya akan "melompat" atau bergeser saat gambar muncul, yang mengganggu kenyamanan pengguna (UX).

2. **Apa perbedaan font CDN dan next/font?**
   - **Font melalui CDN:** Metode ini mengandalkan server pihak ketiga (seperti Google Fonts). Saat halaman dibuka, browser harus melakukan permintaan jaringan (HTTP Request) tambahan ke server luar tersebut. Jika koneksi ke CDN lambat, teks akan muncul dengan font standar sistem terlebih dahulu sebelum berubah ke font asli (Flash of Unstyled Text / FOUT), atau bahkan tidak muncul sama sekali sampai font terunduh (Flash of Invisible Text / FOIT).
   - **next/font:** Fitur ini secara otomatis mengunduh file font pada saat proses build dan menyimpannya secara lokal di dalam aset aplikasi. Keuntungannya adalah tidak ada lagi permintaan ke pihak ketiga, font dimuat secara instan tanpa menghambat proses render halaman, dan privasi pengguna lebih terjaga karena tidak ada pelacakan dari penyedia CDN.

3. **Mengapa script bisa membuat website lambat?**
   Skrip eksternal (JavaScript) secara default bersifat **render-blocking**. Artinya:
   - **Interupsi Parser:** Ketika browser membaca kode HTML dan menemukan tag <script>, browser akan berhenti membaca HTML dan fokus mengunduh serta menjalankan skrip tersebut terlebih dahulu.
   - **Beban Eksekusi:** Skrip yang berat memerlukan daya komputasi CPU yang besar untuk dijalankan. Jika skrip tersebut diletakkan di awal halaman, pengguna akan melihat layar kosong dalam waktu lama karena browser "sibuk" mengolah kode JavaScript sebelum sempat menampilkan elemen visual (seperti teks atau gambar).

4. **Kapan harus menggunakan dynamic import?**
   Dynamic Import harus digunakan untuk memecah kode (*code splitting*) agar browser hanya mengunduh apa yang benar-benar dibutuhkan saat itu juga. Gunakan fitur ini ketika:
   - **Komponen Interaktif yang Tersembunyi:** Seperti Modal, Drawer, atau Dropdown yang hanya muncul jika pengguna mengklik sesuatu.
   - **Library Pihak Ketiga yang Besar:** Misalnya kamu punya halaman admin yang menampilkan grafik menggunakan library berat. Daripada semua pengguna harus mengunduh library grafik tersebut, lebih baik gunakan *dynamic import* agar library hanya diunduh saat user membuka halaman grafik.
   - **Optimasi Jalur Kritis:** Untuk memastikan kode minimal yang dibutuhkan agar halaman depan tampil (LCP) tetap kecil, pindahkan komponen non-kritis ke dalam impor dinamis.

5. Apa dampak bundle size terhadap UX?
   Bundle size merujuk pada total ukuran file JavaScript yang dikirimkan ke browser pengguna. Dampaknya sangat signifikan terhadap pengalaman pengguna:
   - **Interaktivitas (Time to Interactive):** Semakin besar bundle, semakin lama browser memproses JavaScript. Pengguna mungkin sudah melihat tombol di layar, tetapi tombol tersebut tidak bisa diklik karena browser masih sibuk memproses "bundle" yang besar.
   - **Kecepatan Muat pada Jaringan Seluler:** Pengguna dengan koneksi 3G atau 4G yang tidak stabil akan sangat menderita jika harus mengunduh bundle berukuran megabyte. Ini meningkatkan kemungkinan mereka meninggalkan website sebelum halaman terbuka sepenuhnya (*bounce rate*).
   - **Efisiensi Sumber Daya:** Bundle yang kecil membuat aplikasi terasa lebih ringan, responsif, dan hemat baterai pada perangkat seluler.